

Internship Project

Faculty of Science and Engineering
Rijksuniversiteit Groningen

Name:	Stephanie Maya Radhakrishnan
Student Number:	5232546
Research Centre:	Computing Science
Research Project Title:	Data Engineering for Damage Stability Prediction of General Cargo Ships
Primary Supervisor:	Prof. Andrea Capiluppi
Company Supervisor:	Jules Blokhuis, Jan van der Zee

Contents

1	Introduction	1
1.1	Context and background	1
1.2	Problem description	1
1.3	Internship objective	2
1.4	Research questions	2
1.5	Scope and limitations	3
2	Company	4
2.1	Conoship International	4
2.2	SARC and PIAS	4
2.3	SARC & Conoship collaboration in internship	4
3	Project Description	6
3.1	Overall description	6
3.2	Objectives and deliverables	6
3.3	Project timeline	6
3.3.1	Phase 1 - Data extraction and database setup	7
3.3.2	Phase 2 - Data preparation and algorithm identification	7
3.4	AI-controlled damage stability calculations	7
4	Technical Background	8
4.1	Damage stability in ship design	8
4.1.1	Damage stability	8
4.1.2	Probabilistic vs deterministic approach	8
4.2	Damage stability regulations	8
4.3	PIAS software	9
4.4	Hulldf - Hull definition and geometry	9
4.4.1	Purpose	9
4.4.2	Relevant outputs	9
4.5	Layout - Internal subdivision of ship	9
4.5.1	Purpose	9
4.5.2	Relevant outputs	10
4.6	Probdam - Probabilistic damage stability	10
4.6.1	Purpose	10
4.6.2	Relevant outputs	10
4.6.3	Importance	10
4.7	Numerical integration	10
4.7.1	Conventional output - Compartment based	10
4.7.2	Numerical integration method	11
4.7.3	Usage of both	11
4.8	PIAS CLI and XML automation	11
4.9	Relationship between modules	11
4.10	Relevance to AI model training	11

5	Methodology	13
5.1	Overview of data workflow	13
5.2	Obtaining PIAS outputs	14
5.2.1	Automated extraction of Layout XML	14
5.2.2	Manual export of PDF based outputs	14
5.2.3	Numerical integration output	15
5.3	Parsing strategy	15
5.4	Layout XML parsing	15
5.4.1	Parsing structure and categories	15
5.4.2	Subcompartment shapes and frustum geometry	16
5.4.3	Compartment and sub-compartment import	16
5.4.4	Pipe detection	16
5.5	Main dimensions parsing	16
5.6	Openings parsing	16
5.7	Probdam parsing	17
5.7.1	Totals	17
5.7.2	Trim and GM block	17
5.7.3	Conclusion block	17
5.8	Numerical integration parsing	17
5.9	Database structure	17
5.10	Data cleaning and normalization	18
5.11	Versioning and linking across modules	18
5.12	Feature preparation	18
5.13	Reliability and validation	19
6	Results	20
6.1	Internship tasks and achieved results	20
6.2	Overview of imported data	20
6.3	Coverage of ship versions	21
6.4	Compartment and subdivision results	21
6.5	Results for openings	21
6.6	Damage stability results	21
6.7	Summary of dataset completeness	21
6.8	Suitability for machine learning	22
7	AI Algorithm Research	23
7.1	Problem Type and Learning Objective	23
7.2	Nature of the Available Data	23
7.3	Algorithm Families Considered	24
7.3.1	Linear and Logistic Regression	24
7.3.2	Decision Trees and Random Forests	24
7.3.3	Gradient-Boosted Decision Trees	24
7.3.4	Neural Networks (Fully-Connected MLPs)	25
7.3.5	Convolutional Networks on Longitudinal Profiles	25
7.3.6	Graph Neural Networks Using Compartment Connectivity	26
7.4	Recommended Algorithm Family	26
7.5	Use of the SQL Dataset in a Future Model	26
7.6	Summary	26
8	Skills and Knowledge Developed	27
8.1	Technical skills	27
8.2	Domain knowledge	27
8.3	Professional and interdisciplinary skills	27
8.4	Reflection and career impact	27
9	Limitations	28
10	Conclusion	29
11	Future Work	29

1 Introduction

1.1 Context and background

Modern seagoing cargo ships are designed under increasingly strict safety and environmental regulations. One key aspect to ship safety is damage stability: the ability of a vessel to remain afloat and stable after suffering hull damage that leads to flooding of one or more internal compartments. For new build ships, this must be demonstrated rigorously during the design process, typically through a large number of numerical calculations that simulate possible damage scenarios.

Conoship international is a Dutch ship design and engineering company that has been active since 1953 and has contributed to the design of more than 2000 ships. Its work ranges from feasibility studies and consultancy to complete basic and detailed designs, with a focus on efficiency and sustainability in ship operations. To support this design work, Conoship has a dedicated Research and Consultancy department that develops new tools and methods, including data driven approaches.

In this context, Conoship has been using the PIAS ship design software, developed by SARC, since the late 1980s for intact and damage stability calculations, as well as various design tasks. PIAS is a modular naval architecture software suite used worldwide, supporting hull form definition, internal subdivision, hydrostatics, and both deterministic and probabilistic damage stability.

Traditionally, damage stability is conducted by generating many variants of ship design and, for each variant, performing hundreds of damage calculations in PIAS. This process is computationally intensive and time consuming and, more importantly, for this internship, it produces large volumes of technical data scattered across multiple files, formats, and PIAS modules.

At the same time, artificial intelligence (AI) and machine learning are becoming increasingly relevant in maritime applications. For damage stability, a data driven model that can predict the probability that a ship design will pass probabilistic damage stability criteria could significantly accelerate early stage design and reduce repetitive engineering tasks.

However, such a model requires high quality structured data. Conoships projects contain a wealth of relevant information, but this data is not yet available in a unified, machine readable database.

1.2 Problem description

The probabilistic damage stability assessment of general cargo vessels at Conoship is currently based on:

- Ship geometry and subdivision are defined in various PIAS modules (e.g. Hulldef, Layout)
- Damage case generation and evaluation in Probdam (PIAS module for probabilistic damage stability)
- Outputs spread across multiple files (RTF, XML, PDF) and directories

This results in several problems from an AI and data engineering perspective:

- **Fragmented data across modules and formats:** Relevant variables are distributed over Hulldef, Layout, Probdam outputs, stored in different file types, which complicates systematic data collection and reuse.
- **Manual, error prone export procedures:** Historically, exporting PIAS data requires manually triggering output for each ship and each module. This does not scale to the large number of designs and damage cases needed for training machine learning models.
- **Lack of a unified database structure:** There is no central relational database that integrates ship geometry, compartmentalization, loading conditions, damage cases, and pass/fail results for probabilistic damage stability. This prevents efficient querying, analysis, and direct use in AI model training.
- **Unknown data completeness and quality:** Because the data is not stored in a single structured system, it is difficult to assess whether the information for each ship and damage case is complete, consistent and suitable for statistical analysis.

These issues form a bottleneck for Conoship’s ambition to explore AI based tools for damage stability prediction. Before training any model, an underlying data engineering effort is required to extract, clean and structure the data.

Beyond its relevance for future AI model development, the current fragmentation of damage stability data also represents a significant operational inefficiency for Conoship. Engineers and researchers regularly need to retrieve historical damage stability results, compartment definitions, or hydrostatic summaries for existing ship designs. At present, this requires manually navigating multiple project folders, opening PDF and RTF reports from different PIAS modules, and cross-referencing information across files. Depending on the complexity of the request and the age of the project, this process can take tens of minutes to several hours per ship version.

By contrast, the structured SQL database developed in this internship enables instant querying of ship geometry, subdivision data, damage cases, and compliance results. Information that previously required manual searching across multiple files can now be retrieved in seconds through database queries. Even without the use of an AI prediction model, this centralized and machine-readable data structure already provides a substantial time-saving benefit for the company, improving efficiency, reproducibility, and accessibility of damage stability information.

1.3 Internship objective

The main objective of the internship is:

Collect, organize, and structure all relevant cargo ship damage stability data from Conoship’s archives into a PostgreSQL database that can serve as the foundation for training an AI model predicting the probability that a ship will pass the damage stability criteria.

To achieve this, the internship focuses on the following:

- Designing an appropriate SQL schema that captures hull geometry, compartments, loading conditions, damage cases, and probabilistic damage stability results in a relational way.
- Developing automation tools and scripts to extract data from PIAS (via its CLI and module outputs) and import them into the database.
- Performing data cleaning and labeling, including encoding pass/fail outcomes and ensuring that attributes are consistently formatted.
- Preparing the data for use in machine learning, including initial feature structuring and a first assessment of suitable algorithm families.

The result of this internship is not a fully trained and validated AI model, but a robust data foundation and methodological framework that will enable the next steps.

In addition to supporting future AI model development, the resulting database directly improves engineering efficiency by allowing Conoship staff to quickly retrieve and compare damage stability data across ship designs without manual file-based searches.

1.4 Research questions

The work in this internship can be framed around the following main research question:

How can PIAS based damage stability data for general cargo ships be systematically extracted, structured, and stored in a SQL database to support AI based prediction of probabilistic damage stability compliance?

This leads to several sub-questions:

Data extraction

- Which data elements from PIAS modules (Hulldf, Layout, Probdam) are required to describe probabilistic damage stability at the ship and damage case level?

- How can PIAS command line interface and XML capabilities be used to automate bulk data export

Data modeling

How should the SQL schema be structured to represent ships, design variants, compartments, damage cases, and calculation results in a relational and scalable way?

Data preparation

What cleaning, normalization, and labeling steps are necessary to obtain a consistent dataset that can be used by machine learning algorithms?

AI readiness

Given the nature of the available data (types of features, sample size, label distribution), which families of machine learning algorithms appear to be the most suitable for future model development?

1.5 Scope and limitations

The internship is scoped by the following:

- **Ship type:** general cargo ships designed by Conoship, for which probabilistic damage stability has been assessed in PIAS.
- **Software environment:** data generated by PIAS modules, in particular, Hulldef, Layout, and Probdam.
- **Database platform:** PostgreSQL as the target relational database system.

Full development and validation of a production ready AI model is out of scope for this internship period. Instead, the focus is on the data engineering foundation that is necessary before any such AI tools can be reliably developed.

2 Company

2.1 Conoship International

Conoship International is a Dutch ship design and engineering company headquartered in Groningen, with additional offices in Rotterdam and Gdansk. Since 1952, more than 2000 ships have been built according to Conoship design, covering a wide range of vessel types for various sectors of the maritime industry.

Conoship provides services along the full design chain, including:

- Feasibility studies and concept design
- Basic and detail engineering
- Research and consultancy
- Support for ship owners, shipyards, and other maritime stakeholders

The company is known for its emphasis on efficiency and sustainability, for example, by developing fuel efficient hull forms and exploring alternative propulsion concepts. Its research and innovation activities include collaborations with software developers, universities, and other industry partners.

The internship described in this report is carried out within Conoships Research & Consultancy context, at the interface between naval architecture, software tools and data science.

2.2 SARC and PIAS

SARC (Scheepsbouwkundig Advies en Rekencentrum) is a Dutch company that develops software and provides consultancy for naval architecture and ship design. Its flagship product is PIAS (Program for the Integral Approach of Ship design), a modular software suite used by naval architects worldwide for hull form design, stability analysis, and many other tasks.

Conoship has been using PIAS since 1988 to calculate intact and damage stability and to support its ship designs. Over the years, there has been close collaboration between Conoship and SARC, allowing Conoship to provide user feedback and participate in the development of new functionality.

PIAS is structured into modules that each handle a specific aspect of the ship model, for example:

- Hulldef - hull form definition and main ship data
- Layout - design and utilization of the ship's internal layout, including bulkheads, decks, and compartments
- Probdam - probabilistic damage stability calculations
- Additional modules for hydrostatics, loading conditions, tank sounding tables, etc.

Each module can output different types of data, and the software increasingly supports XML based structure output, which are highly relevant for automated data extraction. During the internship, the interaction with SARC focused on clarifying how to use PIAS' command line interface (CLI) and XML exports (in Layout) to systematically access all the needed data.

2.3 SARC & Conoship collaboration in internship

On 28 October 2025, the first official communication between Conoship and SARC was initiated for this internship project. This meeting was held to discuss technical questions related to PIAS and its outputs.

In the meeting, the internship was presented and the objective of constructing the database for AI model training was explained. Possibilities for bulk data extraction from PIAS were discussed, including the availability of a CLI tool capable of exporting data to XML format.

It was confirmed that PIAS could output the Layout data in XML via the CLI enabling automation for that module.

Since the meeting, communication has been through emails in which questions have been asked of the CLI, and outputs.

The conclusion of the meeting was that the questions were resolved, and communication with SARC was initiated for this internship. This confirmed that the planned data engineering approach for the internship was technically feasible and aligned with both Conoship's and SARC's interests.

3 Project Description

3.1 Overall description

The internship focuses on creating the data infrastructure needed for future AI applications at Conoship. The core idea is:

- To collect and organize all relevant PIAS data for a representative set of Conoship designed general cargo vessels.
- To store the data in a structured PostgreSQL database.
- To ensure that the data is clean, labeled, and AI ready, so that machine learning models can later be trained to predict whether a given design will pass the probabilistic damage stability criteria.

The project therefore sits at the intersection of naval architecture, software, data engineering and machine learning.

3.2 Objectives and deliverables

The primary objectives of the internship are described below.

Database population

Populate a PostgreSQL database with all relevant data provided by Conoship for general cargo ship designs, such that the database can be used for AI model training.

SQL schema design

Design a clear, well structured SQL schema that organizes ship data into interrelated tables (e.g. ships, design variants, compartments, openings, damage cases, results), enabling efficient querying and model access.

Data extraction and import scripts

Develop and validate scripts that extract data from PIAS, using automated workflows where necessary, and the PIAS CLI. Parse different file formats and import the parsed data into PostgreSQL.

Data preparation workflow

Implement and document a workflow that covers:

- Data collection from Conoship archives
- Data cleaning (e.g. formatting, unit consistency, removal of irrelevant attributes)
- Data labeling (e.g. pass/fail for damage stability)
- Basic feature engineering for machine learning
- Splitting data into training/validation/test sets (conceptually)

Algorithm recommendations

Based on the nature of the resulting dataset, identify and recommend suitable classes of machine learning algorithms (e.g. logistic regression, tree based methods, gradient boosting, neural networks) that could be used in the follow up AI model development phase.

The final deliverables of the internship are a fully populated PostgreSQL database containing the relevant general cargo ship data from PIAS and a report that describes the database structure and data organization, as well as the extraction cleaning and import methodology. It will also evaluate data completeness and quality and provide recommendations for future AI model training.

3.3 Project timeline

The project plan divides the work into two main phases, originally aligned with the months September, October, and November.

3.3.1 Phase 1 - Data extraction and database setup

September

Familiarization with PIAS and initial data collection

October

- Definition and verification of the SQL schema
- Development of scripts for data extraction from PIAS and data import into PostgreSQL
- Initial execution of scripts on past ship data to verify correct extraction and storage.

3.3.2 Phase 2 - Data preparation and algorithm identification

November

- Completion of data collection and database population for all selected ships
- Data cleaning, labeling, and feature engineering
- First assessment of dataset properties (e.g. size, class balance, feature distributions)
- Literature review and identification of suitable machine learning algorithms for future model training.

This timeline provides structure (with internal deadlines) but was flexible enough to accommodate technical challenges, such as adjustments to the PIAS export setting, handling of complex file formats, or refining the database schema based on practical experience.

3.4 AI-controlled damage stability calculations

The internship is closely related to a broader proposal within Conoship on AI-controlled automation of PIAS damage stability calculations. That proposal envisions an AI agent that would:

- Interface with PIAS to run damage stability calculation automatically
- Assess the results according to relevant regulations
- Propose design modifications when the criteria are not met
- Manage iterative recalculations and design variants

For such an AI agent to be effective, it needs both access to structured historical data, to learn from past designs and their outcomes, and a clear data model describing ships, compartments, and damage cases.

This internship directly contributes to the former, thus building the data backbone that will support future AI experiments. By focusing on general cargo ships and probabilistic damage stability, it provides a concrete and relevant starting point within Conoship's existing project portfolio.

4 Technical Background

This section will provide the theoretical background needed to understand the data engineering tasks performed during the internship. It introduces the principles of damage stability, outlines the relevant regulations, and presents the structure and output of the PIAS software, specifically the Hulldef, Layout and Probdam modules from which data for this project is extracted. The information establishes why certain data elements are necessary, how they relate to each other, and why a structured database is essential for this AI application.

4.1 Damage stability in ship design

4.1.1 Damage stability

Damage stability refers to a ship's ability to remain afloat and maintain sufficient stability after sustaining hull damage that results in the flooding of one or more internal compartments. When a section of the hull is breached, water enters the ship, reducing buoyancy, altering mass distribution, and affecting the vessel's ability to generate restoring moments.

A damage stable ship must satisfy the two following conditions:

- Residual buoyancy: The ship must retain enough intact volume to support its weight even after flooding
- Residual stability: The righting lever curve (GZ curve) must provide sufficient positive area to counter heel and external forces, preventing capsize.

Ensuring this is a critical safety requirement, and international regulations require that vessels, particularly cargo ships and passenger ships, demonstrate a minimum level of survivability when evaluated against various standardized damage scenarios.

4.1.2 Probabilistic vs deterministic approach

Historically, damage stability was evaluated using deterministic rules (such as “flooding in any one compartment and the ship must survive”). Modern regulations, however, use a probabilistic approach as described in the IMO SOLAS Convention because it more accurately reflects real world risk.

In probabilistic damage stability, each potential damage scenario is assigned a probability of occurrence, based on factors such as longitudinal location, height, and extent of damage. For each scenario, the ship is assessed to determine whether it survives or fails (capsizes, sinks, or lacks stability margin). The ship's overall damage stability performance is expressed as an Attained Index A, a weighted sum of all survivable scenarios.

A ship passes the regulation when **Required Index R** $\leq$ **Attained Index A**

Although this internship does not calculate A directly, the underlying output data from PIAS must include the information that determines whether each damage case is considered a pass or fail, since these labels form the target variable for future AI models.

4.2 Damage stability regulations

Although Conoship handles a range of ship types, this internship focuses on general cargo vessels, so we will focus on the regulations relevant to that, which are described in the requirements “Subdivision and Damage Stability” in the SOLAS manual.

The SOLAS probabilistic framework includes definitions for:

- Damage extents in the longitudinal, transverse, and vertical directions
- Survivability calculations for each damage case
- Compartment permeability depending on space type (cargo hold, machinery, accommodation, tanks, etc.)
- Buoyancy and stability criteria after flooding
- Compartment interconnections (which spaces flood together)

PIAS implements these regulations in its Probdam module, automating the generation of damage cases, calculating residual stability curves, and determining whether the ship survives each scenario.

Understanding these inputs and outputs is crucial for designing a database and training an AI model that mimics the pass/fail structure of these criteria.

4.3 PIAS software

PIAS (Program for the Integral Approach of Ship design) is a modular naval architecture suite developed by SARC. It is widely used for conceptual design, hydrostatics, intact stability, and damage stability calculations.

Conoship has been using PIAS since the late 1980s and relies on it in all phases of ship design. The internship work interacts with three main modules:

- Hulldef: defines the hull geometry and main dimensions
- Layout: defines the internal subdivision (compartments, decks, bulkheads)
- Probdam: performs the probabilistic damage stability calculations

Each module contributes different data that must ultimately be combined for machine learning.

4.4 Hulldef - Hull definition and geometry

4.4.1 Purpose

Hulldef is the PIAS module where the fundamental geometry of the vessel is defined. It contains:

- Hull form surface geometry
- Stations, waterlines buttocks
- Main dimensions
- Frame spacing
- References for coordinate systems and hydrostatic origins

This hull model serves as the geometric basis for all other PIAS modules

4.4.2 Relevant outputs

The Hulldef module provides the overall hull geometry and main ship characteristics required for damage stability analysis. Relevant outputs used in this project include the main dimensions of the ship, such as length between perpendiculars (Lpp), length overall (Loa), breadth, and depth. These values are exported from PIAS as PDF reports and form high-level geometric features that remain constant across all damage stability calculations for a given ship version.

4.5 Layout - Internal subdivision of ship

4.5.1 Purpose

The Layout module contains the internal structured subdivision of the ship. It defines:

- Watertight compartments
- Decks, bulkheads, and transverse boundaries
- Connections between compartments

The module is used for damage stability because the extent of flooding depends on compartment boundaries and permeability.

4.5.2 Relevant outputs

The Layout module provides a structured description of the ship’s internal subdivision. Relevant outputs include compartment definitions, subcompartments, permeability values, compartment connectivity, and geometric shape definitions used for flooding calculations. These outputs are exported in XML format and form the basis for representing the internal structure of the ship in the database.

For AI applications, Layout data is particularly important because it defines how flooding propagates through the ship and how internal geometry influences damage stability outcomes.

4.6 Probdam - Probabilistic damage stability

4.6.1 Purpose

Probdam is the PIAS module that implements the probabilistic damage stability requirements of SOLAS. It:

- Generates the set of possible damage scenarios
- Calculates the associated probabilities of occurrence
- Simulates the flooding of relevant compartments
- Computes the resulting hydrostatic condition and righting arm curve
- Determines whether each damage case is a survivable scenario

This module ultimately produces the pass/fail data set labels that the AI model will learn to predict, as well as the attained index A.

4.6.2 Relevant outputs

The Probdam module produces detailed probabilistic damage stability reports. Relevant outputs include per-damage-case boundaries, probabilistic weighting factors, survivability contributions, hydrostatic results under different draft conditions, the attained subdivision index A, the required index R, and the overall compliance result. These outputs are provided as PDF or RTF reports and are parsed into structured database tables.

4.6.3 Importance

Probdam provides:

- The ground truth labels (pass/fail)
- Many of the most important predictive features
- The link between compartment geometry and real hydrodynamic consequences

Thus, the results it provides are the core of any machine learning approach to damage stability.

4.7 Numerical integration

As part of this internship, an additional source of damage stability data is incorporated into the database, a numerically integrated stability output generated alongside the conventional Probdam results. This section explains why both types of output are included and how they complement each other.

4.7.1 Conventional output - Compartment based

The conventional Probdam workflow evaluates damage stability by assigning damage to entire compartments and determining survivability for each scenario. This approach aligns directly with the typical design workflow used at Conoship and the traditional interpretation of SOLAS probabilistic damage stability. In this, damage extents are applied to the compartments and the affected spaces are assumed to be fully flooded. Survivability is checked per damage case and pass/fail output is produced for each case.

This “one-damage-per-compartment” type of calculation is efficient and matches the regulatory compliance process used for ship design. However, geometric resolution is directly related to the compartment breakdown.

4.7.2 Numerical integration method

PIAS also provides an alternative method for evaluating damage cases: the numerical integration method. According to the Probdam module manual, the user may select “numerical integration” as one of the possible methods to calculate the factor *prv* (probability of damage combined with vertical and longitudinal distribution). This method is not explicitly described in SOLAS but is nevertheless a recognized method accepted in the industry. It works with the real shape of the internal geometry instead of relying solely on predefined compartment boundaries and does not use the traditional “damage boundary” concept applied in compartment based calculations.

This is documented in the PIAS manual, there the manual specifies that the method uses a more geometry based integration of the damage region, and that an “Accuracy index numerical integration (0-100)” can be set to control resolution.

Because this approach is less dependent on the discretizations imposed by compartment boundaries, it can provide a more detailed and potentially more accurate representation of how damage extents interact with the ship geometry.

In contrast to the Layout XML export, numerical integration results cannot be generated through the same XML-based command line workflow. Instead, PIAS produces numerical integration output through a dedicated calculation run, which generates an RTF report (outputNumInt.rtf) in the project directory. Because this calculation is computationally expensive, it is treated as a separate, optional step in the data extraction pipeline.

4.7.3 Usage of both

Standard Probdam pass/fail remains the primary regulatory label because it aligns directly with SOLAS interpretation and Conoship’s workflow.

Numerical integration output is collected as secondary, high resolution dataset to enrich the database and to provide better predictive value when used in combination with machine learning models.

This dual method approach helps ensure that the resulting database is complete, flexible, and suitable for both present compliance work and future AI driven tools.

4.8 PIAS CLI and XML automation

PIAS provides a command line interface (CLI) that allows selected modules to be executed and exported automatically. In the context of this internship, the Layout module supports full automation through the CLI and can export structured XML output without manual interaction.

Other modules, such as Probdam (and numerical integration), require initial configuration through the graphical interface before they can be executed automatically via invoke scripts. While this limits immediate large-scale automation, it still allows reproducible batch execution once the setup is complete. Extending full CLI support to these modules would enable complete end-to-end automation of the data extraction pipeline.

4.9 Relationship between modules

Probabilistic damage stability calculation relies on an integrated model composed of the above mentioned modules (Huldef, Layout, Probdam).

In simpler terms, **Huldef** described where the water goes, **Layout** describes what gets flooded, and **Probdam** determines whether the ship survives.

This relationship explains why the SQL database must integrate all three data sources and why a relational schema is necessary.

4.10 Relevance to AI model training

For machine learning, the quality of the input data determines the quality of the predictions. The technical background described above shows that ship level and compartment level geometric data influence flooding.

Also, hydrostatic responses determine survivability damage case definitions provide scenario variation and regulation based probabilities provide scenario weighting.

A future AI model must implicitly learn many of the relationships encoded in PIAS physics based calculations.

Thus, the aim of the technical background is not only to explain how PIAS works but also to justify why the database must be highly structured, why seemingly small geometric details matter, and why careful feature extraction is critical.

5 Methodology

This section describes the methodology used to extract, process and structure the damage stability data used in this project. The goal of the methodology is to convert PIAS outputs into a unified relational database that can support the analysis and future AI model development. The approach combines automated components, manual steps where automation is not yet possible, and a set of dedicated parsing tools designed specifically for this internship. All procedures described reflect the actual workflow implemented during the project.

5.1 Overview of data workflow

The workflow developed in this internship transforms PIAS outputs, produced in XML, PDF, and RTF formats, into a structured PostgreSQL database. Because PIAS is a modular system, the data required for probabilistic damage stability is distributed across several modules: Layout, Probdam, and Hulldef. Each module differs in output format and level of automation support. Figure 1 provides a high-level overview of the data extraction, parsing, and integration workflow implemented during the internship.

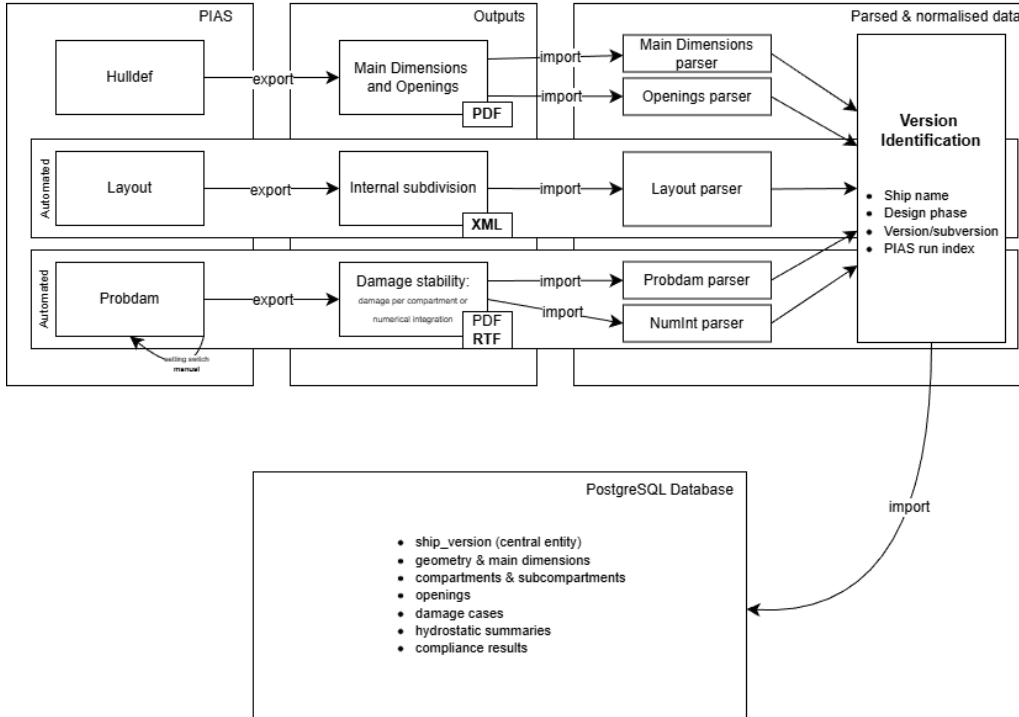


Figure 1: High-level overview of the data extraction, parsing, and database integration workflow

The workflow shown in Figure 1 illustrates how heterogeneous PIAS outputs are transformed into a unified relational database through a combination of automation, dedicated parsers, and consistent version linking.

The main stages of this workflow, summarized in Figure 1, are described in detail below.

1. Obtaining PIAS outputs for each ship version: The Layout module supports full automation through the PIAS command line interface (CLI) and produces structured XML output. The Probdam module, including numerical integration calculations, can also be automated after an initial configuration through the PIAS graphical interface. Hulldef outputs, such as the main dimensions and openings, currently do not support automation and must be exported manually.
2. Parsing module outputs using dedicated Python scripts: Each file type is processed by a dedicated parser. Layout data is parsed from XML, Probdam results are parsed from PDF or RTF files, numerical integration results are parsed from RTF, and Hulldef outputs are parsed from PDF. All parsed data is inserted into the PostgreSQL database using a consistent schema.
3. Integrating outputs using a consistent versioning scheme: All outputs are linked using a filename and directory based versioning strategy that identifies ship name, design phase, version, subversion, and run

index. This ensures that the results of different PIAS modules are correctly associated with the same ship version.

4. Automated and semi-automated execution of PIAS calculations: Layout exports are executed fully automatically via the CLI. Probdam (damage per compartment and numerical integration) calculations are executed using automated invoke scripts once the required PIAS menu settings have been configured. This separation reflects current tooling limitations rather than fundamental restrictions in the workflow.

This approach ensures that each ship version can be represented as a coherent and extensible dataset combining geometry, subdivision, openings, and damage stability results.

5.2 Obtaining PIAS outputs

5.2.1 Automated extraction of Layout XML

The Layout module is the only PIAS module that supports automated export through the command line interface. During the internship, a Python script (`runInvoke.py`) was developed to automatically generate Layout XML files for a large number of existing ship projects.

The script scans the project folder structure and identifies ship designs based on known design phase folders (such as Concept Design, Basic Design, and Final Design). From the directory structure, the script determines the ship name, design phase, version number, and subversion. Different internal naming styles (for example v2, run1, or numbered folders) are normalized to a consistent numeric version format.

For each detected ship version, the script generates the required PIAS request files and executes PIAS via an automatically generated `invoke_run.bat`. The resulting Layout output is saved using a standardized filename of the form:

`ship_design_version_subversion_piasShip.fromLayout.xml`

Although the file content is XML, the `.fromLayout` extension is used consistently to distinguish Layout exports from other PIAS outputs.

This naming convention is used throughout the project and allows all module outputs to be linked reliably during parsing. Once the Layout XML file is generated, it is immediately processed by the Layout parser and imported into the PostgreSQL database without any manual steps.

The automation workflow uses existing Probdam output files to detect ship versions and PIAS run identifiers, after which the Layout XML export is triggered automatically for the corresponding ship configuration.

5.2.2 Manual export of PDF based outputs

Some PIAS outputs required for this project are not yet available through direct XML-based command line export and therefore rely on either manual export or partial automation. These outputs include the Probdam report, the Main Dimensions report, and selected Hulldef outputs.

The Probdam module produces probabilistic damage stability results in PDF or RTF format. While these calculations can be executed automatically after a one-time configuration through the PIAS graphical interface, the initial setup introduces a delay and prevents immediate large-scale execution across all ship versions. Once configured, the resulting reports can be parsed automatically without further user intervention.

Main dimensions are generated from the Hulldef module and are only available as PDF outputs. Hulldef does not currently support automated export through the PIAS CLI, and these reports must therefore be exported manually for each ship version.

Openings data conceptually belongs to the Hulldef module; however, because Hulldef exports are not automated, openings are currently extracted from the Layout XML instead. The Layout module contains a subset of opening information and can be exported fully automatically. This approach prioritizes automation and consistency across ship versions, even though the Layout-based opening data does not include all Hulldef specific attributes.

Although some outputs require manual steps or initial configuration, all files follow a strict naming convention. Once the files are available in the input directory, the parsing and database insertion process is fully automated and reproducible.

5.2.3 Numerical integration output

In addition to the conventional probabilistic damage stability results produced by the Probdam module, PIAS can generate numerical integration based damage stability results. These calculations are executed within the Probdam module as a dedicated calculation run and produce a textual report (outputNumInt.rtf) in the corresponding ship version directory.

Calculations for numerical integration can be automated after an initial configuration using the PIAS graphical interface. Once this configuration is in place, numerical integration runs can be triggered automatically using invoke scripts, similar to standard Probdam calculations. However, because numerical integration calculations are computationally expensive and the setup cannot currently be configured through the PIAS command line interface, this step is treated as optional in large-scale batch execution.

When numerical integration is enabled, the calculation is executed automatically after the standard Probdam run has completed. The resulting outputNumInt.rtf file is detected and parsed using a dedicated parser, and all extracted results are inserted into the PostgreSQL database. If the output file already exists, the numerical integration step can be skipped to avoid unnecessary recomputation.

Although numerical integration results are not yet available for all versions of the ships, the database schema and parsing workflow fully support these results. Once numerical integration setup and execution are exposed through the PIAS CLI, this step can be executed efficiently across the entire ship archive.

5.3 Parsing strategy

Each file type required its own dedicated parser, because the formats differ between PIAS modules. The parsing strategy emphasized strict consistency (ensuring that every ship version is treated in the same way) and preservation of the structure present in PIAS outputs.

All parsers:

- Connect directly to the PostgreSQL database
- Parse one file at a time
- Insert records in a transactional manner
- Close the connection after each run

In addition to XML and PDF parsers, a dedicated RTF parser is implemented for numerical integration output, following the same transactional insertion and version linking strategy as the Probdam parser.

5.4 Layout XML parsing

The Layout parser processes the .fromLayout.xml file and extracts internal subdivision data. Layout contains the full watertight subdivision model used by PIAS for stability calculation, and therefore forms one of the most important components of the database.

5.4.1 Parsing structure and categories

Layout XML files contain hierarchical blocks that describe content categories, compartments, subcompartments, and geometric definitions. The parser first imports all content categories into the content_category table using an ON CONFLICT DO NOTHING rule to avoid duplicates across ship versions.

5.4.2 Subcompartment shapes and frustum geometry

For each subcompartment shape defined in the Layout XML, the parser stores the shape identifier, the side of the ship, and the associated geometric definition. Shapes of type frustum are defined by a set of frustum points, each containing longitudinal (L), transverse (B), and vertical (H) coordinates.

All frustum points belonging to a shape are imported into the database. In addition, the parser computes an approximate span of each shape in the transverse and vertical directions by taking the difference between the maximum and minimum B and H values. These span values provide a compact description of the size of the shape and are later used during pipe detection.

This approach keeps the original geometric detail from PIAS while also providing simplified geometric measures that are useful for further analysis.

5.4.3 Compartment and sub-compartment import

Only compartments marked as “selected for output” in PIAS are imported. This matches the workflow inside PIAS, where only activated compartments contribute to the subdivision model. For each imported compartment, all subcompartments are processed and linked to the appropriate shape GUID.

5.4.4 Pipe detection

Pipes are not explicitly labeled as a separate object type in the Layout XML. For this reason, pipe detection is implemented as part of the parsing process using a simple heuristic based on naming conventions and geometry.

A subcompartment is classified as a pipe when:

- the name of the parent compartment contains the word “pipe”
- the name does not contain “pipeduct”
- and the geometric span of the associated shape in either the breadth (B) or height (H) direction is smaller than 0.3 m.

This rule combines information from the compartment name with geometric properties derived from the Layout model. The resulting pipe classification is stored as a boolean flag in the database and can be used directly as an input feature for later machine learning experiments.

5.5 Main dimensions parsing

The main dimensions parser extracts the overall dimensions of the ship from the first page of the main dimensions PDF. These values are found through regular expressions that match PIAS’ printed dimension format. Once extracted, they are inserted into the main_dimensions table.

These values are important because they provide high level geometric features that remain constant across all damage stability calculations for a given ship version.

5.6 Openings parsing

The openings parser processes the tabulated data in the openings report PDF. The parser extracts opening descriptions, dimensions, opening type and the compartment to which the opening is connected.

The parser accounts for variable PDF table formats by skipping header rows, ignoring rows without content, and interpreting the table column by column. When possible, openings are linked to compartments already imported from the Layout XML. This linking is based on the compartment names appearing directly in the report.

Although openings are conceptually defined within the Hulldef module, they are currently extracted from the Layout XML. This is a deliberate design choice made to prioritize automation, as the Layout module supports fully automated export via the PIAS CLI, while Hulldef does not. The Layout-based opening data does not contain all Hulldef-specific attributes, but it provides sufficient geometric and structural information to be used as input features for future machine learning models. This approach ensures consistent coverage across ship

versions while maintaining a fully automated extraction pipeline.

The outputs of this parser populates the opening table.

5.7 Probdam parsing

The Probdam parser extracts probabilistic damage stability results from PIAS output reports. These reports are available as RTF or PDF files and contain several different types of information spread across multiple pages.

Damage cases are identified by numerical case identifiers (for example “1.1” or “2.3”). For each damage case, the parser extracts the damage boundaries, probabilistic factors for different draft conditions, and the corresponding contribution to the subdivision index. Aggregate rows starting with “Total” are detected separately and stored as summary values.

In addition to the damage case data, the parser extracts hydrostatic summary values for three draft conditions: light service draft, partial subdivision draft, and deepest subdivision draft. For each condition, draft, trim, GM, displacement, and subdivision index values are stored. Finally, the conclusion section of the report is parsed to obtain the required index, attained index, and the overall compliance result.

These outputs provide both detailed damage case information and ship-level results that are required for later analysis and model training.

5.7.1 Totals

The row starting with “Total” aggregated probabilistic values across the longitudinal extents and are inserted into `probdam_total`.

5.7.2 Trim and GM block

The second to last page of the Probdam report contains hydrostatic summaries for three draft conditions: light, partial, and deepest. The parser extracts for each condition: draft, trim, VCG, MG, displacement, attained subdivision index, and required index.

Because each version of the ship contains exactly one entry per condition, the table `trim_gm` uses a unique constraint on (`ship_version_id`, `condition_name`) and an ON CONFLICT update rule.

5.7.3 Conclusion block

The final page provides the overall regulatory outcome. The parser extracts the subdivision length, required index R, attained index A, and the compliance flag. These values are stored in `probdam_conclusion`.

5.8 Numerical integration parsing

Numerical integration output is parsed from the `outputNumInt.rtf` file generated by PIAS. The structure of this report differs from the Probdam output and therefore requires a separate parser. The numerical integration parser extracts per-damage-case results, including probabilistic contributions under different draft conditions and the resulting contribution to the attained subdivision index.

In addition to per-case values, the parser extracts summary information from the conclusion section of the report, including the subdivision length, required subdivision index R, attained subdivision index A, and the overall compliance result. Where available, additional metadata such as numerical integration step accuracy and penetration reference settings are also stored.

All numerical integration data is linked to the corresponding ship version using the same filename and directory-based versioning strategy that is used throughout the project.

5.9 Database structure

All parsed data is inserted into a PostgreSQL database whose schema is defined in `shipData.sql`. The schema reflects the structure of PIAS and enforces the relationships necessary for damage stability analysis.

The key design features include:

- A separate `ship_version` table to unify all outputs belonging to the same design iteration
- Foreign keys connecting compartments, openings, damage cases, and hydrostatic summaries to their ship version
- Normalization of geometric and structural entities into separate tables
- Boolean attributes for flags such as `selected_for_output`, `is_pipe`, and `complies`
- Indexes to improve query performance across large fleets

5.10 Data cleaning and normalization

During parsing, several measures were taken to ensure clean and consistent data:

- The values of “INF” and “-INF” in XML files were mapped to SQL NULL
- Text fields were trimmed to remove whitespace
- Numeric types were validated using explicit float or integer conversion
- Only compartments explicitly selected in PIAS were included
- Duplicate content categories and trim/GM rows were resolved using primary key and unique constraints
- Rows in PDF tables that lacked numerical content were ignored

These steps preserve the numerical integrity of the dataset and ensure that all the derived features are based on complete and valid information.

5.11 Versioning and linking across modules

PIAS output files do not include explicit identifiers that link results from different modules to the same ship version. To address this, a consistent filename based versioning approach is used throughout the project.

Each ship version is represented by a single entry in the `ship_version` table. The identifying information (ship name, design phase, version, subversion, and run index) is derived from the file name and folder structure. This information is extracted in the same way by all parsers.

Using this approach ensures that Layout XML files, Probdam reports, openings data, and main dimensions are all linked to the correct ship version. It also allows new data to be added later without changing the database structure or parsing logic.

In addition to version and subversion identifiers, the PIAS internal ship run identifier is stored to preserve the distinction between multiple calculation runs of the same ship version. While ship versions are uniquely identified using ship name, design phase, version, and subversion, the ship run value provides additional traceability to the original PIAS calculation.

5.12 Feature preparation

Although the internship does not include training a machine learning model, several features were prepared during parsing to support future data driven analysis. These include:

- Breadth and height spans used for pipe detection
- Opening dimensions in metric units
- Damage boundaries and probabilistic factors
- Hydrostatic summaries under different drafts
- Compliance labels and subdivision index values

All of these fields are stored directly in the database, allowing them to be retrieved efficiently for ML pipelines.

5.13 Reliability and validation

The reliability of the methodology was ensured by iterative testing. The parsed entries were regularly checked against the PIAS reports to confirm numerical correctness. Because the schema enforces foreign key relationships, invalid combinations of ship version and structural elements cannot be inserted. This gives a strong guarantee that parsed data accurately represents the information contained in the original PIAS outputs.

6 Results

This section presents the results of the data extraction, parsing, and database population workflow developed during the internship. Because the primary objective of the project was to build a structured and reliable dataset rather than to train a predictive model, the results focus on the completeness, consistency, and structure of the imported PIAS data, as well as the characteristics of the resulting database.

6.1 Internship tasks and achieved results

The internship focused on building a structured and automated data foundation for probabilistic damage stability analysis at Conoship. The tasks performed and results achieved during the internship can be summarized as follows.

First, a relational database schema was designed and implemented in PostgreSQL to integrate heterogeneous data originating from multiple PIAS modules. This schema captures ship versions, hull geometry, compartment and subcompartment structures, openings, damage cases, hydrostatic summaries, and regulatory compliance results in a unified and scalable way.

Second, automated and semi-automated data extraction workflows were developed. A fully automated command-line workflow was implemented for the Layout module, enabling bulk export of XML-based internal subdivision data across many ship versions. For Probdam and numerical integration calculations, automated execution scripts were developed that allow batch processing after an initial one-time configuration through the PIAS graphical interface. Manual export remains necessary for Hulldef-derived main dimensions due to current software limitations.

Third, dedicated parsing tools were implemented for each output format used by PIAS, including XML, PDF, and RTF. These parsers extract relevant numerical and structural data and insert it into the PostgreSQL database using a consistent versioning strategy. This ensures that outputs from different modules are correctly linked to the same ship version.

As a result of these tasks, a populated and validated database was produced that integrates geometry, subdivision, openings, hydrostatic properties, and probabilistic damage stability results for a representative set of Conoship-designed general cargo vessels. The resulting dataset is structurally complete, reproducible, and suitable for future machine learning applications aimed at predicting probabilistic damage stability compliance.

6.2 Overview of imported data

The methodology described in Section 5 resulted in a populated PostgreSQL database that integrates data from multiple PIAS sources:

- Layout XML
- Probdam (RTF or PDF)
- Numerical Integration RTF
- Openings (Layout XML; Hulldef PDF when available)
- Main Dimensions PDF

Each parser contributed to one or more tables in the database, all linked through the `ship_version` entity. The result is a coherent data set in which the geometric subdivision, openings, hydrostatic characteristics, and damage stability results for each version of the ship are stored together.

Across all ships processed during the internship, the following high level outcomes were achieved:

- Each ship version imported into the database contains entries in the `compartment`, `subcompartment`, `opening`, `probdam_case`, and `main_dimensions` tables
- All Layout XML files were successfully parsed and produced complete compartment and subcompartment structures
- Probdam reports were consistently parsed, capturing damage cases, and summary metrics

- Opening reports were parsed with partial linking, where compartment names appeared directly in the PDF table
- Main dimensions were extracted for some ship versions with available PDFS

These results confirm that the pipeline successfully integrates data across PIAS modules despite differences in format and automation availability.

6.3 Coverage of ship versions

Because PIAS projects are organized by design phase and version, the number of ship versions available varies across Conoship's project archive. Using the version identification approach described in Section 5.10, all Layout, Probdam, and Hulldef files with consistent naming were linked to ship versions.

6.4 Compartment and subdivision results

The Layout parser produces a detailed structural model for each ship version. A typical ship version contains dozens of compartments and several hundred frustum points describing the internal geometry.

6.5 Results for openings

The opening parser imports all openings reported in the PIAS PDF. The number of openings per ship version varies depending on the level of detail included in the design.

Because linking openings to compartments depends on the presence of compartment names in the PDF, the database contains both linked and unlinked openings. This is expected given PIAS' formatting. All openings, whether linked or unlinked, remain useful as geometric features.

6.6 Damage stability results

The Probdam parser produced the most consequential results for potential AI applications. For each ship version, the database now contains the following:

- A complete set of damage cases extracted from PIAS
- Probabilistic factors
- Survivability contributions (ai)
- Hydrostatic summary values for three draft conditions
- The final attained index A and required index R
- A binary compliance flag

These fields form the basis for future predictive modeling.

In addition to the conventional Probdam results, numerical integration outputs are available for selected ship versions. While Probdam results are used as the primary regulatory ground truth, numerical integration data provides a complementary, higher-resolution perspective on damage stability behavior and may serve as an additional source of predictive features in future AI models.

6.7 Summary of dataset completeness

The parsing and database integration steps led to a complete multi-module dataset for every ship version for which all four PIAS outputs were available. The following observations summarize the dataset's completeness:

- Layout XML coverage is high due to CLI automation
- Openings and Probdam PDFs are available for most ship versions, but must be manually exported
- Linking across modules succeeds consistently because filename patterns are uniform
- Missing or malformed rows in PDF tables are gracefully handled through filtering

- Hydrostatic summaries always produce exactly three draft conditions when included in the report
- Numerical integration results are available for a subset of ship versions. The reduced coverage is due to current automation limitations and computational cost. Once numerical integration setup and execution are exposed through the PIAS CLI, these calculations can be run efficiently for all ship versions

Therefore the dataset is structurally complete, even if individual ship versions differ in the number of compartments, openings, or damage cases.

6.8 Suitability for machine learning

The final result of this methodology is a structured relational dataset containing:

- geometric variables
- compartmental subdivision
- opening geometries
- probabilistic damage stability values
- compliance labels

This ensures that the database is ready for future AI experimentation. The presence of continuous and categorical variables provides a rich basis for feature engineering. The compliance label stored in `probdam_conclusion` serves directly as the target variable for supervised learning.

7 AI Algorithm Research

This section investigates suitable classes of machine learning (ML) algorithms for predicting probabilistic damage stability outcomes using the structured dataset produced in this internship. Although the development of a trained AI model is outside the scope of this internship, the database assembled in previous sections provides the foundation for such work. The goal is to evaluate which ML approaches are appropriate given 1) the structure and content of the PIAS-derived data, 2) the expected dataset size, and 3) the underlying physical characteristics of probabilistic damage stability.

7.1 Problem Type and Learning Objective

The future AI model aims to approximate the regulatory outcome of a probabilistic damage stability calculation. Two closely related prediction tasks can be identified:

- **Binary classification:** predict whether a given ship version complies with SOLAS probabilistic damage stability requirements (target: `probdam_conclusion.complies`).
- **Regression:** predict the attained subdivision index A, a continuous value that sums the overall survival of the ship (target: `probdam_conclusion.attained_index`).

Both prediction tasks fall under **supervised learning**, where the model learns from historical examples of ship versions whose PIAS-calculated outcomes are stored in the PostgreSQL database. Because the underlying physics are nonlinear (flooding interactions, hydrostatics, permeability effects, GM/trim interactions), the choice of algorithm must be able to capture complex relationships while remaining robust to limited sample sizes.

7.2 Nature of the Available Data

The database integrates a wide range of structured information:

- **Scalar geometric features:** main dimensions such as Lpp, Loa, breadth, depth, and hydrostatic descriptors such as GM, trim, VCG, and displacement.
- **Categorical attributes:** content categories, compartment types, and other labels that describe the function of spaces.
- **Structural counts:** the number of compartments, openings, pipes, and other discrete elements per ship version.
- **Aggregated geometric descriptors:** spans of frustum shapes in transverse and vertical directions, longitudinal distributions of compartments, and similar derived quantities.
- **Damage stability outputs:** per-damage-case contributions a_i , aggregated totals, the final attained index A, the required index R, and the binary compliance flag.

Each ship has multiple design versions, and the current dataset already includes several ships, with more expected. This places the likely data volume in the range of tens to a few hundred ship versions initially, eventually growing further, plus thousands of individual damage case records. For ML purposes, this is considered a small-to-medium sized tabular dataset.

Therefore the algorithm family must:

- work well with heterogeneous tabular features,
- handle nonlinear interactions between variables,
- remain stable with a limited but growing sample size, and
- provide some degree of interpret-ability, useful for engineering insight and design decisions.

7.3 Algorithm Families Considered

7.3.1 Linear and Logistic Regression

Linear regression (to predict A) and logistic regression (to pass/fail) are classical statistical models. They assume:

- approximately linear relationships between features and outputs, and
- limited interaction effects between variables.

Advantages:

- high interpretability
- extremely fast training and easy implementation
- useful as a baseline for comparison

Limitations:

- unable to model strong nonlinearities, such as interactions between geometry, openings, and hydrostatics
- typically under perform on engineered datasets with complex underlying physics

Assessment: linear and logistic regression are useful as reference models but are unlikely to provide sufficient predictive performance for damage stability outcomes.

7.3.2 Decision Trees and Random Forests

Decision trees partition the feature space by simple threshold rules. Random forests combine many trees trained on different subsets of the data.

Advantages:

- naturally model nonlinear relationships
- handle mixed feature types (numeric and categorical)
- relatively robust to overfitting
- provide measures of feature importance

Limitations:

- individual trees can be unstable
- large forests may be harder to interpret at a detailed level
- performance often plateaus before that of more advanced tree-based boosting methods

Assessment: random forests constitute a strong baseline for tabular engineering datasets and are well-suited to the structured ship data, but they are typically outperformed by gradient-boosted tree models.

7.3.3 Gradient-Boosted Decision Trees

Gradient-boosted decision trees build a model by sequentially adding small trees, each correcting the errors of the previous ensemble.

Advantages:

- state-of-the-art performance on tabular data
- excellent behavior on small-to-medium datasets (from tens up to tens of thousands of samples)
- capture complex interactions, such as how GM, trim, and longitudinal volume distribution jointly affect A, or how openings and compartment geometry interact
- handle missing values and heterogeneous features with minimal preprocessing

- provide feature importance measures, supporting interpretability and engineering insight

Limitations:

- more hyper parameters than linear models or simple trees, requiring tuning
- do not directly encode geometric structure unless this is captured in engineered features

Assessment: gradient-boosted decision trees are the most suitable algorithm family for this project. They match the data regime and physical complexity of probabilistic damage stability calculations and are widely used in industrial practice for similar tabular prediction problems.

7.3.4 Neural Networks (Fully-Connected MLPs)

Fully-connected neural networks (multi layer perceptrons) can, in principle, approximate any function given sufficient capacity and data.

Advantages:

- highly flexible model class
- capable of representing complex nonlinear functions

Limitations:

- typically require much larger datasets than currently available
- sensitive to feature scaling and noise
- prone to overfitting in small engineering datasets
- less interpretable than tree-based models and require more training effort

Assessment: fully-connected neural networks are not recommended as the primary approach for the expected dataset size. They are likely to under perform compared to gradient boosted trees while being harder to train and explain.

7.3.5 Convolutional Networks on Longitudinal Profiles

If the ship geometry were converted into a regularized longitudinal “image” or 1D profile, convolutional neural networks (CNNs) could be explored.

Advantages:

- can capture spatial patterns along the ship length
- may align better with how damage propagates longitudinally

Limitations:

- require substantial preprocessing and engineering of input representations
- need relatively large datasets to perform reliably
- significantly more complex to implement and tune

Assessment: CNN based approaches are interesting for future research, but they are not practical for the near term model given the current scope and data volume.

7.3.6 Graph Neural Networks Using Compartment Connectivity

Data at the compartment level naturally form a graph, with compartments as nodes and bulkhead or flooding connections as edges. Graph neural networks (GNNs) could, in principle, model damage stability by propagating information across this graph.

Advantages:

- closely aligned with the physical structure of compartments and flooding paths
- promising for high fidelity modeling of individual damage cases and flooding interactions.

Limitations:

- require complex graph construction and a more advanced ML framework
- benefit from large, diverse datasets

Assessment: GNNs are a strong candidate for long term research, particularly if the goal is to predict per damage case quantities. However, they are not necessary for predicting ship level compliance metrics in the first stage of AI development.

7.4 Recommended Algorithm Family

Given the characteristics of the dataset assembled in this internship structured tabular features, nonlinear physical relationships, and a small to medium number of ship versions, the recommended algorithm family is:

Gradient Boosted Decision Trees

This choice is motivated by:

- high performance on tabular engineering datasets
- strong handling of nonlinearity and feature interactions
- robustness to limited but growing dataset sizes
- compatibility with heterogeneous attributes and missing values
- availability of feature importance measures
- relatively low demands on data preprocessing

As the database grows with additional ships and design versions, the performance of boosted tree models is expected to improve further, while the underlying assumptions remain valid.

7.5 Use of the SQL Dataset in a Future Model

A future ML pipeline based on the current database would follow these high level steps:

1. **Feature extraction:** query and aggregate ship level and compartment level information into a fixed feature vector for each ship version.
2. **Label preparation:** select either the attained index A (for regression) or the compliance flag (for binary classification) from the damage stability results.
3. **Model training:** train a gradient boosted decision tree model using cross validation to estimate generalization performance.
4. **Interpretation:** analysis of feature importance to understand which aspects of geometry, subdivision, openings, and hydrostatics most influence the predicted damage stability outcome.

This pipeline directly supports engineering decisions and can be integrated into future AI-controlled damage stability workflows.

7.6 Summary

The algorithm research carried out in this section concludes that **tree based gradient boosting** provides the optimal combination of predictive power, robustness, interpretability, and practicality for the probabilistic damage stability prediction task. More advanced neural architectures (such as CNNs or GNNs) may become relevant in later research phases, particularly if the goal shifts towards modeling individual damage cases or flooding paths in more detail. For the current stage, however, gradient boosted decision trees offer a well founded and realistic path forward for AI-based prediction of damage stability compliance.

8 Skills and Knowledge Developed

This internship contributed significantly to the development of both technical and domain-specific skills, as well as professional competencies relevant to interdisciplinary engineering projects.

8.1 Technical skills

From a computer science and data engineering perspective, the internship strengthened skills in relational database design, including schema normalization, foreign key relationships, and performance oriented indexing. Practical experience was gained in developing robust data parsing pipelines for complex and heterogeneous file formats such as XML, PDF, and RTF. In addition, automation skills were developed through the use of command-line interfaces, batch execution scripts, and reproducible data workflows.

The project also required careful handling of data quality, including validation, normalization, and consistency checks. These tasks reinforced best practices in data engineering, particularly in the context of preparing datasets for downstream machine learning applications.

8.2 Domain knowledge

The internship provided in-depth exposure to naval architecture concepts, specifically probabilistic damage stability and its regulatory framework under SOLAS. Through working with PIAS outputs, a practical understanding was developed of how hull geometry, internal subdivision, openings, and hydrostatic properties influence survivability calculations. This knowledge was essential for determining which data elements are relevant for AI-based prediction and how they should be structured in a database.

8.3 Professional and interdisciplinary skills

Working within Conoship's Research & Consultancy environment required close collaboration with naval architects and interaction with an external software developer at SARC. This strengthened the ability to communicate technical data engineering concepts to domain experts and to translate engineering requirements into software solutions. The internship also improved problem solving skills in situations where tooling limitations required pragmatic design choices, such as balancing automation against manual steps.

Overall, the internship reinforced the ability to work at the interface of engineering, software systems, and data-driven methods, which is essential for applied AI projects in industrial contexts.

8.4 Reflection and career impact

This internship provided valuable insight into the practical challenges of applying data-driven methods within an engineering-intensive industrial environment. One of the main learning experiences was understanding how complex engineering tools such as PIAS generate large volumes of data that are not immediately suitable for machine learning. Bridging this gap required not only technical data engineering skills but also a deep understanding of the underlying physical and regulatory principles.

The project highlighted the importance of automation and reproducibility in large-scale engineering workflows, as well as the limitations that arise when legacy software does not fully support modern data extraction practices. Addressing these constraints required careful design decisions and close communication with domain experts, reinforcing the importance of interdisciplinary collaboration.

From a career perspective, the internship confirmed an interest in working at the intersection of computer science, engineering, and applied artificial intelligence. It demonstrated how data engineering forms a critical foundation for AI applications and how such work can directly support decision-making in complex technical domains such as ship design. The experience has provided a strong basis for pursuing future work in applied data science or AI-driven engineering systems, either in industry or in further academic research.

9 Limitations

The scope and outcomes of this internship are subject to several limitations, primarily related to tooling constraints, data availability, and project boundaries.

First, the level of automation achievable in the data extraction pipeline is constrained by the current capabilities of the PIAS command line interface. While the Layout module supports fully automated XML export, other modules such as Probdam and numerical integration require an initial one-time configuration through the graphical user interface before automated execution is possible. Additionally, Hulldef derived outputs, including main dimensions, do not currently support command-line export and therefore require manual generation. These limitations affect execution efficiency but do not impact the correctness or reproducibility of the extracted data once available.

Second, coverage of numerical integration results is incomplete across the full set of ship versions. Numerical integration calculations are computationally expensive and, due to current configuration and automation constraints, were only executed for a subset of the available designs. As a result, the dataset contains uneven coverage between conventional probabilistic damage stability results and numerical integration outputs. The database schema and parsing workflow fully support numerical integration data, and this limitation is therefore operational rather than structural.

Third, the size of the dataset is limited by the number of available Conoship-designed general cargo vessels and their associated design versions. While sufficient for establishing a robust data engineering pipeline and conducting preliminary algorithm selection, the dataset is not yet large enough to support reliable training and validation of complex machine learning models. Expanding the dataset will be necessary before predictive performance can be evaluated quantitatively.

Finally, the internship deliberately excludes the implementation, training, and validation of an AI model. The focus of the project is on creating a structured, high-quality data foundation suitable for future machine learning applications. Model development, hyperparameter tuning, and performance evaluation are therefore left as future work.

These limitations reflect practical constraints of the software environment and the defined scope of the internship, rather than fundamental weaknesses in the methodology or data design.

10 Conclusion

This internship focused on building the data engineering foundation required for future AI-driven prediction of probabilistic damage stability at Conoship. The work resulted in a structured, scalable PostgreSQL database that integrates PIAS outputs from multiple modules, including Layout, Probdam, and Hulldef derived reports. This integration was achieved despite heterogeneous file formats, partial automation capabilities, and the absence of built-in version identifiers in PIAS outputs.

A key outcome of the internship is a robust and reproducible data pipeline that transforms PIAS results into machine-readable relational data. Layout data is extracted fully automatically via the PIAS command line interface, while Probdam and numerical integration calculations can be automated after initial configuration through the graphical interface. Although Hulldef outputs such as main dimensions still require manual export, consistent naming conventions and dedicated parsers ensure that all data sources are reliably linked at the ship-version level.

The resulting database captures both high-level ship characteristics, such as main dimensions and hydrostatic summaries, and detailed internal structure, including compartment geometry, openings, and per-damage-case stability results. Numerical integration data is fully supported in the schema and parsing workflow and is already used where available, with coverage currently limited only by automation and computational constraints.

The database was validated for consistency and completeness and can now be queried systematically, enabling analyses that were previously impractical due to fragmented data storage. By storing geometric features, probabilistic damage stability results, and compliance labels in a unified structure, the dataset is directly suitable for future machine learning applications.

Overall, this internship delivers a complete and extensible data engineering framework that removes a major barrier to AI-assisted damage stability assessment at Conoship. With further automation support from PIAS, particularly through extended CLI functionality for Probdam and numerical integration, the pipeline can be scaled efficiently across entire ship archives, providing a strong foundation for predictive modeling in early-stage ship design.

11 Future Work

The next step following this internship is the development and implementation of the AI model that will predict probabilistic damage stability outcomes based on the structured database created. With the data foundation now complete, future work will focus on selecting, training, and validating a gradient-boosted decision tree model, integrating feature extraction pipelines, and evaluating predictive accuracy against PIAS results. Once implemented, such a model can be incorporated into Conoship's design workflow to support early-stage decision making and accelerate iterative damage stability assessments.

A key technical recommendation emerging from this internship is to extend PIAS command line interface support to additional modules, in particular Probdam and numerical integration calculations. While these calculations can already be automated after manual configuration through the graphical interface, exposing all relevant settings and execution options through the CLI would enable fully automated large-scale damage stability analysis across entire ship archives. This would significantly reduce execution time, remove remaining manual bottlenecks, and allow the complete database pipeline developed in this internship to operate in a fully automated manner.